

Lab Practice Exercises 3

Lecture 5 · Practice only — nothing here is collected or graded

Fall Semester 2026 · Dr. Narasimha Rao Vajjhala · nrao@aubg.edu · Office MB 253

BEFORE YOU START

Environment	JDK 25 (the current Long-Term Support release) and IntelliJ IDEA — the same setup as Labs 1 and 2. Check it with <code>javac -version</code> before Question 1.
What this covers	Lecture 5: defining and calling methods, return types, pass by value, overloading, scope and stepwise refinement — the only session of Week 3, since Tuesday 22.09 was Independence Day. The loops and conditionals inside the questions are Week 2's.
Feeding the samples in	Every sample is a whole console session. Put the input in a text file and redirect it: <code>java Program.java < input.txt</code> . Questions 14 and 18 are two files each: keep both in one folder and run the one with <code>main</code> — since JDK 22 the launcher finds the other file by itself.
Style	Give every method a <code>/** */</code> comment — one sentence saying what it returns, and an <code>@param</code> line for each parameter, as Lecture 5's practice slide asks. Brace every <code>if</code> and loop body, as in Lab 2.
Nothing to hand in	These exercises are for practice. There is nothing to submit, no deadline and no mark — keep your <code>.java</code> files somewhere you can find them again, and bring anything that will not work to the lab or to office hours.

Questions 1 to 20 are the core set. Each names the headers it wants: write the header first, then the body, then a small `main` that calls it with a few inputs — the order the lecture's practice slide asks for. Between them they finish that slide's list: `isPrime`, `reverse` and the palindrome test in Questions 3 and 4, the triangle in 5, `Hex2Dec` in 13, the calendar in 20 and `countPrimes` in 27. Questions 21 to 30 are about the rules the core set leans on without naming them: the return rule, pass by value, overloading, scope, and the call stack.

None of this is collected, so work at whatever pace suits you and skip anything you already find easy. Two suggestions all the same. Predict the compiler's verdict before you compile anything in Part B — most of the value is in being right in advance. And reuse methods rather than retyping them: `isPrime` turns up in Questions 3 and 18 and the leap-year test in 19 and 20, and every copy is a copy you can get wrong. Question 27 is about exactly that.

Quiz 1 on Thursday 01.10 covers Weeks 1 to 3, methods included; predicting what code prints before you run it, as Questions 23, 24 and 29 ask, is the best practice here for it. Questions 13 and 18 call `length()` on a `String`, which is Thursday's material — the lecture's `Hex2Dec` slide uses the same call, with `charAt`, and those two are all you need.

Part A · Core Exercises

Question 1 · Digit Sum and Digital Root

Write a method `public static int sumDigits(long n)` that returns the sum of the decimal digits of a non-negative number, and a method `public static int digitalRoot(long n)` that keeps summing digits until one digit is left — 987654321 sums to 45, and 45 sums to 9. The program reads one non-negative number and prints both.

`digitalRoot` must call `sumDigits` rather than contain a digit loop of its own. The parameter is a `long` because the input may be longer than an `int` can hold.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a non-negative integer: 9876543210123
Sum of digits: 51
Digital root: 6
```

For every n above 0 the digital root is $1 + (n - 1) \% 9$. Write a second class, `DigitalRootCheck`, whose `main` — a driver, in the lecture's word — compares your method with that formula for every n up to 1,000,000 and prints the number of disagreements. It should print 0; a million checks in under a second is what a driver buys you.

Question 2 · Course Grades and Grade Point Average

The COS 2030 syllabus grades on this scale: 96–100 A, 90–95.99 A-, 86–89.99 B+, 83–85.99 B, 80–82.99 B-, 76–79.99 C+, 73–75.99 C, 70–72.99 C-, 65–69.99 D+, 60–64.99 D, below 60 F. Write `public static String letterGrade(double score)` for it, and `public static double gradePoints(String letter)` for the usual 4.0 scale: A 4.00, A- 3.67, B+ 3.33, B 3.00, B- 2.67, C+ 2.33, C 2.00, C- 1.67, D+ 1.33, D 1.00, F 0.00.

The program reads the number of courses, then the credits and the percentage for each, prints the letter and grade points as it goes, and finishes with the credit-weighted GPA to two decimal places.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the number of courses: 4
Credits and score for course 1: 3 91.5
  A- (3.67)
Credits and score for course 2: 3 84
  B (3.00)
Credits and score for course 3: 4 77.25
  C+ (2.33)
Credits and score for course 4: 2 95.99
  A- (3.67)
GPA over 12 credits: 3.06
```

Weighted, not plain: a 4-credit C+ pulls harder than a 2-credit A-. Check the sample by hand — if you get 3.17, you averaged the four grade points and left the credits out. And make `gradePoints` a switch expression whose `default` throws, so that a typo such as "a-" fails loudly instead of scoring zero.

Question 3 · Emirps: Primes That Reverse into Other Primes

An emirp — prime spelt backwards — is a prime whose digits, reversed, form a different prime: 13 is one, because 31 is prime too, and 11 is not, because its reversal is itself. Write `public static boolean isPrime(int n)`, `public static int reverse(int n)` and `public static boolean isPalindrome(int n)`, and use all three to print the first K emirps, ten per line.

`isPrime` is the lecture's version, which returns the moment it finds a divisor; `reverse(3456)` is 6543, as on the practice slide.

EXPECTED OUTPUT (EXAMPLE)

```
Enter K: 30
13 17 31 37 71 73 79 97 107 113
149 157 167 179 199 311 337 347 359 389
701 709 733 739 743 751 761 769 907 937
```

`isPalindrome` should be a single line that calls `reverse`. If yours has a loop of its own, you have written `reverse` twice.

Question 4 · Reverse and Add

Take a number, add it to its reversal, and repeat until the result reads the same both ways: $87 + 78 = 165$, $165 + 561 = 726$, $726 + 627 = 1353$, $1353 + 3531 = 4884$ — four steps. Print every step, and give up after 30.

The sums can outgrow an `int` quickly — from 196, the 18th sum is already past `Integer.MAX_VALUE` — so overload your Question 3 methods for `long`: `public static long reverse(long n)` and `public static boolean isPalindrome(long n)`. Keep the `int` versions in the class as well. The names stay the same; the compiler picks a version from the type of the argument.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a number: 87
87 + 78 = 165
165 + 561 = 726
726 + 627 = 1353
1353 + 3531 = 4884
Palindrome after 4 steps
```

Run it with 196, the smallest number for which nobody has ever found a palindrome this way. After 30 steps the sum has 16 digits, and the 40th addition would pass `Long.MAX_VALUE` and turn negative — so the step limit is also what keeps the arithmetic honest.

Question 5 · Triangle Toolkit

Write `public static boolean isValid(double a, double b, double c)`, true when each side is shorter than the sum of the other two, and `public static double area(double a, double b, double c)` using Heron's formula: with $s = (a + b + c) / 2$, the area is the square root of $s(s - a)(s - b)(s - c)$. These two are the lecture's `MyTriangle`.

Add `public static String classify(double a, double b, double c)`, returning "equilateral", "isosceles" or "scalene", and `public static boolean isRight(double a, double b, double c)`. The program reads three sides and reports all of it, or says that they cannot form a triangle.

EXPECTED OUTPUT (EXAMPLE)

```
Enter three sides: 3 4 5
Triangle: scalene, right-angled
Area: 6.00
```

```
Enter three sides: 1 2 3
Not a triangle
```

`isRight` cannot test `a * a + b * b == c * c` on doubles. For the right triangle 0.03, 0.04, 0.05 the left side comes to 0.0025 and the right to 0.0025000000000000005, so the test says no. Compare the difference with a small tolerance instead — and find the longest side first, because nobody promises to type it last.

Question 6 · Fractions with Euclid's Algorithm

Lecture 5 turned Lab 2's GCD loop into a method that tries every k up to the smaller number. Euclid's algorithm is far faster: `gcd(a, b) = gcd(b, a % b)`, repeated until the second number is 0, when the first one is the answer. Write it as `public static int gcd(int a, int b)` with a `while` loop, and `public static int lcm(int a, int b)`, which calls it.

Use both to add two fractions $a/b + c/d$: bring them to the least common denominator, add, and reduce. Print every stage.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a/b + c/d as four integers: 1 6 3 10
1/6 + 3/10 = 5/30 + 9/30 = 14/30 = 7/15
```

Count the passes each version makes for `gcd(1836311903, 1134903170)` — two consecutive Fibonacci numbers, Euclid's worst case: 44 for Euclid, 1,134,903,169 for the lecture's loop. Same answer, same header. The caller cannot tell which body it got, which is the point of a method.

Question 7 · Loan Repayment Schedule

A loan of P euros at an annual rate of r percent, repaid monthly over n years, costs $P \cdot i / (1 - (1 + i)^{-12n})$ a month, where $i = r / 1200$ is the monthly rate. Write `public static double monthlyPayment(double principal, double annualRate, int years)` and `public static void printSchedule(double principal, double annualRate, int years, int months)`, which prints the first months rows of the repayment table.

Each payment first covers that month's interest on the outstanding balance; the rest of it repays principal. The program reads the loan, the rate, the years and how many months to show.

EXPECTED OUTPUT (EXAMPLE)

```
Loan amount (EUR): 10000
Annual interest rate (%): 6
Years: 3
Months to show: 3
Monthly payment: 304.22
Month  Interest  Principal  Balance
   1      50.00    254.22    9745.78
   2      48.73    255.49    9490.29
   3      47.45    256.77    9233.52
Total repaid over 3 years: 10951.90
```

Two things to try. Show all 36 months: the last balance prints as `-0.00`, a few ten-billionths of a euro below zero — rounding error left over from 36 steps, which `printf` rounds to `0.00` without dropping the sign. Then try a rate of 0: the formula divides 0 by 0 and the payment comes out as `NaN`. Decide what the method should do instead, and say so in its `/** */` comment.

Question 8 · Square Roots by Repeated Averaging

The Babylonian method finds a square root by averaging: start with any positive guess — 1.0 will do — and replace it with $(\text{guess} + n / \text{guess}) / 2$ until the new guess equals the old one. Write `public static double sqrt(double n)` for $n > 0$, and `public static void printRow(double n)`, which prints n , your square root, `Math.sqrt(n)`, and whether the two are the same double.

Call `printRow` for 2, 3, 10, 144, 0.0625 and `1e10`. Where the last column says false, which of the two values is right? Read what the documentation of `Math.sqrt` promises before you answer.

EXPECTED OUTPUT

	n	sqrt(n)	Math.sqrt(n)	same?
2.0	1.414213562373095	1.4142135623730951	false	
3.0	1.7320508075688772	1.7320508075688772	true	
10.0	3.162277660168379	3.1622776601683795	false	
144.0	12.0	12.0	true	
0.0625	0.25	0.25	true	
1.0E10	100000.0	100000.0	true	

Try 0 and -4 before you trust it. For 0 the guess halves until it underflows to 0.0, the next step computes 0.0 / 0.0, and the guess becomes NaN — which is not equal even to itself, so the loop never ends. For -4 there is no root to settle on at all. Deal with both before the loop, and say in the javadoc what the method returns for them.

Question 9 · Credit Card Number Validation

The last digit of a card number is a check digit (Luhn's algorithm). Double every second digit from the right, adding the two digits of any result above 9; add the other digits unchanged; the total must be a multiple of 10. The number must also have 13 to 16 digits and start with 4, 5, 37 or 6.

The design is fixed — here are the seven headers. Write the bodies, and let `isValid` do nothing but combine the other six.

THE SEVEN HEADERS

```
public static boolean isValid(long number)           // the whole test
public static int sumOfDoubleEvenPlace(long number) // 2nd, 4th, ... digits, doubled
public static int getDigit(int number)              // one digit, or the sum of two
public static int sumOfOddPlace(long number)        // 1st, 3rd, ... digits
public static boolean prefixMatched(long number, int d) // does number start with d?
public static int getSize(long d)                  // how many digits d has
public static long getPrefix(long number, int k)    // its first k digits
```

EXPECTED OUTPUT (EXAMPLE)

```
Enter a card number as a long integer: 4388576018410707
4388576018410707 is valid

Enter a card number as a long integer: 4388576018402626
4388576018402626 is invalid
```

Build it from the bottom up: a four-line driver for `getDigit` — 0, 7, 12 and 18 should give 0, 7, 3 and 9 — before `isValid` exists. With seven methods, a card that fails tells you only that one of them is wrong; a driver that fails tells you which. Test with made-up numbers such as the two in the sample, never with a real card.

Question 10 · Sales Commission and a Target Search

A sales firm pays monthly commission in tiers: 8% of the first 5,000 EUR of sales, 10% of the next 5,000, and 12% of everything above 10,000. Write `public static double commission(double sales)`.

Then use it the other way round: read a target commission, and find the smallest whole-euro sales figure that earns at least that much, by calling `commission` inside a loop.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the target commission (EUR): 2500
Smallest sales figure: 23334 EUR, earning 2500.08 EUR
```

Test the boundaries before you search: `commission(5000)` must be 400.0 and `commission(10000)` 900.0. A method you have tested is a method you can call twenty thousand times without looking inside it.

Question 11 · Collatz Records

Starting from any positive n , halve it if it is even and replace it with $3n + 1$ if it is odd; every start anyone has tried reaches 1 in the end. Write `public static long next(long n)`, `public static int steps(long n)` — how many applications of `next` it takes to reach 1 — and `public static long peak(long n)`, the highest value on the way.

The program reads a limit and reports which start below it takes the most steps, and the highest value that any of them reaches.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the limit: 1000000
Most steps below 1000000: 837799 (524 steps)
Highest value reached: 56991483520 (starting from 704511)
```

Change every `long` to `int` and run it again. At 113,383 the sequence climbs past `Integer.MAX_VALUE`, $3n + 1$ wraps round to a negative number, and a negative number never reaches 1 — the program does not crash; it simply never finishes. That is why the parameters are `long`.

Question 12 · Day of the Week by Zeller's Congruence

Zeller's congruence gives the day of the week of any Gregorian date: $h = (q + 26(m + 1)/10 + k + k/4 + j/4 + 5j) \% 7$, with integer division throughout, where q is the day of the month, m the month, $j = \text{year} / 100$ and $k = \text{year} \% 100$. January and February count as months 13 and 14 of the previous year. The result is 0 for Saturday, 1 for Sunday, and so on to 6 for Friday.

Write `public static int dayOfWeek(int year, int month, int day)` and `public static String dayName(int h)`, a switch expression, and print the day for a date the user types, in the Bulgarian `day.month.year` form.

EXPECTED OUTPUT (EXAMPLE)

```
Enter year, month and day: 2026 9 29
29.09.2026 is a Tuesday
```

```
Enter year, month and day: 2027 1 1
01.01.2027 is a Friday
```

Test 1 January and 1 March 2026 — a Thursday and a Sunday. The January–February adjustment is where Zeller goes wrong, and a date in September never exercises it. Notice, too, that `dayOfWeek` is free to change its own `month` and `year`: they are copies, and the caller's variables are untouched.

Question 13 · Hexadecimal in Both Directions

Complete the lecture's `Hex2Dec` (slide 27) and extend `hexCharToDecimal` so that it accepts lower-case a–f as well and returns -1 for any other character; `hexToDecimal` then returns -1 for a string that is not hexadecimal, the empty string included. Add Lab 2's Question 9 as a method, `public static String decimalToHex(int n)`.

The program converts one hexadecimal number the user types, then runs a driver loop that sends every `n` from 0 to 65535 through `decimalToHex` and back through `hexToDecimal`, and counts the failures.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a hex number: 7e3
The decimal value of 7e3 is 2019
Round trips from 0 to 65535: 0 failures
```

```
Enter a hex number: 12G4
12G4 is not a hexadecimal number
Round trips from 0 to 65535: 0 failures
```

A round trip can pass with both methods wrong in matching ways. If `decimalToHex(0)` returns an empty string — Lab 2's defect — and `hexToDecimal("")` returns 0, as the lecture's loop does, the driver reports no failure. With the -1 for an empty string it reports one; check 0 by eye all the same. And try `0x1F`: the `x` is not a hexadecimal digit, so the answer is -1; Java's own `Integer.parseInt("0x1F", 16)` rejects it too.

Question 14 · Random Characters from Another Class

Type the lecture's `RandomCharacter` class (slide 28) into `RandomCharacter.java` — four methods and no `main`. In the same folder, write `RandomCharacterTest.java`, whose `main` calls `RandomCharacter.getRandomLowerCaseLetter()` 26,000 times and counts how often 'a', 'm' and 'z' come up, then prints a ten-character password of alternating upper-case letters and digits built by `public static String password(int length)`.

Run it with `java RandomCharacterTest.java`. Each count should be near 1,000, and yours will differ from the sample on every run.

EXPECTED OUTPUT (ONE RUN)

```
java RandomCharacterTest.java
26000 random lower-case letters:
'a' came up 1020 times
'm' came up 1002 times
'z' came up 962 times
Password: T6E7A9R1P2
```

Now delete the + 1 from `getRandomCharacter` and run it again: 'z' never comes up, and nothing else looks wrong. That one character is the difference between 26 letters and 25.

Question 15 · Monthly Sales Bar Chart

Write `public static void printBar(String label, double value, double unit)`, which prints the label, one # for every unit of value (rounded to the nearest whole bar) and the value itself, and `public static String monthName(int month)`, a switch expression returning "Jan" to "Dec".

The program reads six monthly sales figures from one line and prints a bar for each, at 100 EUR per #, with the values lined up in one column.

EXPECTED OUTPUT (EXAMPLE)

```
Enter sales for January to June (EUR): 1250 980 1745 2210 1980 2400
Jan | #####                1250
Feb | #####                980
Mar | #####                1745
Apr | #####                2210
May | #####                1980
Jun | #####                2400
Each # is 100 EUR
```

`"#".repeat(Math.round(value / unit))` does not compile: `Math.round(double)` returns a `long`, and `repeat` takes an `int`. The compiler reports possible lossy conversion from `long` to `int`. A method's return type is part of its header, and Question 29 has more of these.

Question 16 · Invoice Amount in Words

Invoices and cheques state the amount in words as well as in figures. Write `public static String inWords(int n)` for any whole number from 0 to 999,999, built from smaller methods — `below20`, `tens`, `below100` and `below1000` are one way to divide it — so that 1234 becomes "one thousand two hundred thirty-four".

The program reads an amount in euros and prints it in words, cents included. Use no "and" inside a number: the only "and" is the one before the cents. One euro and one cent are singular.

EXPECTED OUTPUT (EXAMPLE)

```
Enter an amount (EUR): 1234.56
one thousand two hundred thirty-four euros and fifty-six cents
```

```
Enter an amount (EUR): 1001.01
one thousand one euros and one cent
```

Get the cents with `Math.round(amount * 100)`, not with a cast. The double nearest to 19.99 is a hair below it, so `19.99 * 100` is 1998.999999999998 and `(int) (19.99 * 100)` is 1998 — a cent lost on the most ordinary price there is.

Question 17 · Converting Leva Prices to Euro

Bulgaria adopted the euro on 1 January 2026 at the fixed rate of 1.95583 leva to the euro. Write `public static double roundToCent(double amount)` and `public static double toEuro(double leva)`, which converts at that rate and rounds to the nearest cent; keep the rate in one named constant.

The program reads prices in leva until 0, prints each one converted, and then the total two ways: the converted prices added up, and the prices added up in leva and then converted.

EXPECTED OUTPUT (EXAMPLE)

```
Enter prices in leva, 0 to finish: 0.99 0.99 0.99 12.50 0
0.99 BGN = 0.51 EUR
0.99 BGN = 0.51 EUR
0.99 BGN = 0.51 EUR
12.50 BGN = 6.39 EUR
Converted one by one, then added: 7.92 EUR
Added in leva, then converted:    7.91 EUR
```

The two totals differ by a cent, and neither is wrong: each rounding moves a price by up to half a cent, and here the three 0.99s all moved the same way. A program has to decide where the rounding happens — and if that decision lives in one method, changing it is a one-line edit.

Question 18 · A Utility Class of Number Tests

Collect number tests in a class of their own, `NumberUtils.java`, with no main: `isPrime`, `reverse` and `isPalindrome` from Question 3; `sumProperDivisors` and `isPerfect` ($28 = 1 + 2 + 4 + 7 + 14$); and `countDigits`, `power` and `isArmstrong` ($153 = 1^3 + 5^3 + 3^3$).

Then write `Classify.java`, which reads numbers until 0 and prints the properties of each, calling every method as `NumberUtils.isPrime(n)` and so on — exactly as you have been calling `Math.sqrt` since the first week.

EXPECTED OUTPUT (EXAMPLE)

```
java Classify.java
Enter numbers, 0 to finish: 153 28 131 8128 97 100 7 0
153: Armstrong
28: perfect
131: prime, palindrome
8128: perfect
97: prime
100: none of these
7: prime, palindrome, Armstrong
```

Compile only `Classify.java`, with `javac Classify.java`, and list the folder: `NumberUtils.class` is there too, because `javac` compiled the class it needed. Then run `javadoc -d docs NumberUtils.java` and open `docs/index.html` — your `/** */` comments are now a web page.

Question 19 · Date Utilities

Write `public static boolean isLeapYear(int year)`, `public static int daysInMonth(int year, int month)` (a switch expression with comma-separated labels), `public static boolean isValidDate(int year, int month, int day)` and `public static int dayOfYear(int year, int month, int day)`, where 1 January is day 1.

The program reads a date, typed day first as Bulgarians write it, and prints its position in the year and how many days of the year remain — or reports that the date does not exist.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a date (day month year): 29 9 2026
29.09.2026 is day 272 of 2026; 93 days of the year remain

Enter a date (day month year): 29 2 2027
29.02.2027 is not a valid date
```

Every date method in this lab takes year, month, day, in that order — as `dayOfWeek` in Question 12 and the calendar in Question 20 do. With parameters of one type, one fixed order is the only defence: swap two of them and it still compiles, the lecture's warning. So resist `daysBetween(y1, m1, d1, y2, m2, d2)`; turn each date into a number of days from a fixed starting day and subtract, as `getTotalNumOfDays` does in Question 20.

Question 20 · Printing a Month Calendar

Implement the whole of the lecture's `PrintCalendar` structure chart (slide 29): `printMonth`, `printMonthTitle`, `printMonthBody`, `getMonthName`, `getStartDay`, `getTotalNumOfDays`, `getNumOfDaysInMonth` and `isLeapYear`. `getStartDay` counts the days since 1 January 1800, which was a Wednesday.

The program reads a year and a month and prints that month's calendar.

EXPECTED OUTPUT (EXAMPLE)

```
Enter full year (e.g., 2026): 2026
Enter month as a number between 1 and 12: 9
    September 2026
-----
Sun Mon Tue Wed Thu Fri Sat
      1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30
```

Two checks. The chart's `readInput` box cannot be one method — a method returns one value, and there are two to read — so read them in `main`, or write two small methods. And `getStartDay` and Question 12's Zeller compute the same thing by different routes: a driver that compares them for every month from 1800 to 2100 should report 3,612 months and no disagreements. Zeller counts from Saturday.

Part B · Additional Exercises

These questions cover the rules of Lecture 5 that Part A leans on without naming them. Most ask you to record what the compiler or the runtime actually printed — copy those messages exactly, character for character, rather than paraphrasing them. Every fragment is a method, or a few statements, inside an otherwise complete class: compile each one as a class of its own.

B1 · THE RETURN RULE

The compiler checks every path out of a method, and it judges by the shape of the code, not by the values your variables could ever take. Each fragment below is legal Java apart from that one rule — or so it seems.

Question 21 · Five Methods and the Return Rule

Predict which of the five fragments compile, then compile each one as a class of its own and copy the messages.

THE FIVE FRAGMENTS

```
// (a) a for loop that always finds a divisor
public static int firstDivisor(int n) {
    for (int d = 2; d <= n; d++) {
        if (n % d == 0) {
            return d;
        }
    }
}
```

(a) The author reasoned that every n from 2 up has a divisor between 2 and n , so a `return` is always reached. The compiler disagrees. What does it not know — and what did the author forget? What is the smallest change that satisfies it?

```
// (b) the same search as a while (true) loop
public static int firstDivisor(int n) {
    int d = 2;
    while (true) {
        if (n % d == 0) {
            return d;
        }
        d++;
    }
}
```

(b) The `while (true)` version compiles. Why can the compiler prove this one? Now add `if (d > n) { break; }` inside the loop and compile again.

```
// (c) a switch statement with a return in every case
public static double points(char grade) {
    switch (grade) {
        case 'A': return 4.0;
        case 'B': return 3.0;
        case 'C': return 2.0;
        case 'D': return 1.0;
        case 'F': return 0.0;
    }
}

// (d) the same mapping as a switch expression
public static double points(char grade) {
    return switch (grade) {
        case 'A' -> 4.0;
        case 'B' -> 3.0;
        case 'C' -> 2.0;
        case 'D' -> 1.0;
        case 'F' -> 0.0;
    };
}
```

(c) Every grade has its `return`. Record the message, and count how many of the possible values of a `char` the switch handles.

(d) The same mapping as a switch expression. Record its message. Which of (c) and (d) names the real problem, and what one line fixes each?

```
// (e) one line too many
public static int twice(int n) {
    return n * 2;
    System.out.println("doubled " + n);
}
```

(e) Why is this an error rather than a warning, when the extra line would do no harm?

Question 22 · void, Values, and Values Thrown Away

A void method performs an action; a value-returning method produces a value. The compiler keeps the two apart — except in one direction, and that is the one to watch. Here `printGrade` is the lecture's void method and `square(int n)` returns `n * n`.

THE FRAGMENTS

```
// (a)
int grade = printGrade(85);

// (b)
System.out.println(printGrade(85));

// (c)
public static void report(double score) {
    System.out.println("Score: " + score);
    return score >= 60;
}

// (d)
public static int square(int n) {
    if (n < 0) {
        return;
    }
    return n * n;
}

// (e)
square(7);
Math.max(3, 9);
System.out.println("done");
```

Compile (a) to (d), each in a class of its own, and copy the four messages. For each one, say in a sentence which promise in the method's header the code breaks.

(e) compiles and prints only done: a value-returning method called as a statement simply has its result thrown away. When is that exactly what you want — and when is it a bug that `javac` will never report?

B2 · PASS BY VALUE

Java copies every argument into its parameter. A method can do what it likes to its copies and the caller never sees it — which is why, until next week's arrays, the only way to get a value out of a method is return.

Question 23 · Three Methods That Change Nothing

CHANGENOTHING.JAVA

```
public class ChangeNothing {
    public static void main(String[] args) {
        int x = 1;
        int y = 2;
        swap(x, y);
        double score = 78.5;
        addBonus(score, 5);
        int count = 42;
        reset(count);
        System.out.println(x + " " + y + " " + score + " " + count);
    }

    public static void swap(int a, int b) {
        int temp = a;
        a = b;
        b = temp;
    }

    public static void addBonus(double score, double bonus) {
        score = score + bonus;
    }

    public static void reset(int counter) {
        counter = 0;
    }
}
```

- (a) Predict the one line this prints, then run it.
- (b) Draw the stack at the moment `addBonus` is about to return, in the style of the lecture's call-stack slide: every frame, every variable, every value.
- (c) Rewrite `addBonus` so that `main` gets the new score. Why is the same repair impossible for `swap`?
- (d) Rename `addBonus`'s parameter from `score` to `mark` and run it again. What changes, and why was the shared name never the problem?

B3 · OVERLOADING

The compiler chooses an overload from the argument types alone — an exact match first, then the nearest widening — and it chooses before the program runs. It does not always choose what the author meant.

Question 24 · Which Overload Runs?

WHICH.JAVA

```
public class Which {
    public static void main(String[] args) {
        byte small = 5;
        show(7);
        show(7L);
        show(7.0f);
        show('7');
        show(small);
        show(7 / 2);
        show(7 / 2.0);
        show(3_000_000_000L);

        System.out.println(money(1250));
        System.out.println(money(1250.0));
    }

    public static void show(int n)    { System.out.println("int    " + n); }
    public static void show(long n)   { System.out.println("long   " + n); }
    public static void show(double n) { System.out.println("double " + n); }

    /** Formats a whole number of cents: 1250 gives "12.50 EUR". */
    public static String money(long cents) {
        return String.format("%.2f EUR", cents / 100.0);
    }

    /** Formats an amount in euros: 1250.0 gives "1250.00 EUR". */
    public static String money(double euros) {
        return String.format("%.2f EUR", euros);
    }
}
```

(a) Write down the ten lines you expect, then run it. Explain every surprise from Java's widening conversions: byte → short → int → long → float → double, and char → int.

(b) `money(1250)` was written by someone who meant 1,250 euros. What happened, and why is renaming the two methods — `moneyFromCents` and `moneyFromEuros` — a better fix than remembering to write `1250.0`?

Question 25 · When the Compiler Refuses

Compile each fragment below as a class of its own and copy the four messages.

THE FOUR FRAGMENTS

```
// (a)
public static void main(String[] args) {
    System.out.println(total(1, 2));
}

public static long total(int a, long b) {
    return a + b;
}

public static long total(long a, int b) {
    return a + b;
}
```

(a) Which line does javac point at — the definitions or the call? What does that tell you about when an ambiguity is detected?

```
// (b)
public static int half(int n) {
    return n / 2;
}

public static double half(int n) {
    return n / 2.0;
}

// (c)
public static double area(double width, double height) {
    return width * height;
}

public static double area(double height, double width) {
    return width * height;
}

// (d) only max(int, int) and max(double, double) exist
System.out.println(max(1, 2, 3));
```

(b) Two methods that differ only in their return type.

(c) Two methods that differ only in their parameter names. What do (b) and (c) together tell you about which parts of a header make up its signature?

(d) Read this message to the end: it lists every candidate and the reason each was rejected. Which line of it would you read first?

Finally, fix (a) twice: once with a cast at the call, once by adding an exact overload. Which fix would you rather find in someone else's code?

B4 · SCOPE

A local variable lives from its declaration to the end of the block that contains it, and it must be given a value before it is read. Four of the five fragments below break one of those two rules.

Question 26 · Five Fragments, Five Messages

Predict which of the five fragments compiles, then compile each as a class of its own and copy the five messages.

THE FIVE FRAGMENTS

```
// (a)
public static void a() {
    for (int i = 0; i < 3; i++) {
        int square = i * i;
    }
    System.out.println(i + " " + square);
}

// (b)
public static void b(int n) {
    int n = 10;
    System.out.println(n);
}

// (c)
public static void c() {
    int total = 0;
    for (int k = 1; k <= 3; k++) {
        int total = k;
    }
    System.out.println(total);
}
```

- (a) Two errors from a single line: why two?
- (b) A parameter and a local variable with one name.
- (c) The same name declared inside the loop.

```
// (d)
public static void d() {
    for (int k = 1; k <= 3; k++) { System.out.print(k); }
    for (int k = 1; k <= 3; k++) { System.out.print(k * 10); }
    System.out.println();
}

// (e)
public static int larger(int a, int b) {
    int result;
    if (a > b) {
        result = a;
    } else if (b >= a) {
        result = b;
    }
    return result;
}
```

(d) This declares `k` twice in one method and compiles. State in one sentence the rule that separates (c) from (d).

(e) The return rule in another form. Fix it with the smallest change, and say why the compiler cannot see that `a > b` and `b >= a` between them cover every case.

Lecture 5's case for methods was one copy of each idea. These two questions are that case, run as experiments.

Question 27 · Three Copies, One of Them Wrong

The program below counts the primes in three ranges with three copies of one loop. It compiles, it runs, and it prints plausible numbers. One copy is wrong.

PRIMEREPORT.JAVA

```
public class PrimeReport {
    public static void main(String[] args) {
        int count = 0;
        for (int n = 2; n <= 100; n++) {
            boolean prime = true;
            for (int d = 2; d * d <= n; d++) {
                if (n % d == 0) { prime = false; break; }
            }
            if (prime) { count++; }
        }
        System.out.println("Primes from 2 to 100: " + count);

        count = 0;
        for (int n = 101; n <= 1000; n++) {
            boolean prime = true;
            for (int d = 2; d * d < n; d++) {
                if (n % d == 0) { prime = false; break; }
            }
            if (prime) { count++; }
        }
        System.out.println("Primes from 101 to 1000: " + count);

        count = 0;
        for (int n = 1001; n <= 10000; n++) {
            boolean prime = true;
            for (int d = 2; d * d <= n; d++) {
                if (n % d == 0) { prime = false; break; }
            }
            if (prime) { count++; }
        }
        System.out.println("Primes from 1001 to 10000: " + count);
    }
}
```

(a) Run it. Is anything about the output suspicious? There are 1,229 primes below 10,000.

(b) Extract `public static boolean isPrime(int n)` and `public static int countPrimes(int from, int to)`, so that `main` shrinks to three calls, and run it again. Which copy was wrong, what was the difference, and which numbers did it count that it should not have?

(c) Add one line to `main` that checks the three parts add up to `countPrimes(2, 10000)`. That line is a driver in miniature, and it would have caught the bug on the first run.

Question 28 · Calling a Method in Another Class — and a Name to Avoid

Put the lecture's `MyMath.java` and `Report.java` (slide 15) in one folder, and add `System.out.println(Math.sqrt(2));` to `Report's` main.

- (a) Compile only `Report.java`, with `javac Report.java`, and list the folder. How many `.class` files are there, and why?
- (b) Run `javadoc -d docs MyMath.java` and open `docs/index.html`. What does `javadoc` warn about, and which of its warnings can you fix today?
- (c) Now rename `MyMath` to `Math` — the class and the file — and change `MyMath.max` to `Math.max` in `Report`. Compile again and copy the message. Which `Math` did the compiler use for `Math.max`, which for `Math.sqrt`, and what does that tell you about naming your own classes?

B6 · HEADERS AND STACK TRACES

A method's header is its contract, and a stack trace is the call stack printed at the moment something broke. Both reward slow reading.

Question 29 · The Header Is the Contract

HEADERCONTRACT.JAVA

```
public class HeaderContract {
    public static void main(String[] args) {
        System.out.println(Math.round(2.5));
        System.out.println(Math.round(-2.5));
        System.out.println(Math.round(2.5f));
        System.out.println(Math.abs(-7));
        System.out.println(Math.abs(Integer.MIN_VALUE));
        System.out.println(Math.max(3, 4.0));
        System.out.println(Math.pow(2, 10));
        System.out.println(Math.sqrt(-1));
        System.out.println(-7 % 3);
        System.out.println(Math.floorMod(-7, 3));
    }
}
```

- (a) Predict all ten lines. For each surprise, find the method's header and description — hover over the call in IntelliJ, or look up `java.lang.Math` in the Java documentation — and explain the output from those alone.
- (b) Add `int r = Math.round(2.5);` and copy the compiler's message. Then change `2.5` to `2.5f`. Why does that compile?
- (c) `Math.abs(Integer.MIN_VALUE)` is negative. Find the sentence in the documentation of `Math.abs(int)` that says so.

Question 30 · Reading a Stack Trace

GRADEREPORT.JAVA

```
public class GradeReport {
    public static void main(String[] args) {
        printSummary(870, 0);
    }

    public static void printSummary(int totalPoints, int students) {
        System.out.println("Average: " + average(totalPoints, students));
    }

    public static int average(int total, int count) {
        return total / count;
    }
}
```

EXPECTED OUTPUT — PART (A)

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at GradeReport.average(GradeReport.java:11)
    at GradeReport.printSummary(GradeReport.java:7)
    at GradeReport.main(GradeReport.java:3)
```

(a) Read the trace from the top: which method was running when the exception happened, who called it, and from which line? Draw the three frames in the style of the lecture's call-stack slide, and label each with its at line.

SELF CALL.JAVA

```
public class SelfCall {
    public static void main(String[] args) {
        System.out.println(average(3, 4));
    }

    /** Returns the average of two ints, as a double. */
    public static double average(int a, int b) {
        return average(a, b);
    }

    /** Returns the average of two doubles. */
    public static double average(double a, double b) {
        return (a + b) / 2;
    }
}
```

(b) This `average(int, int)` was meant to hand its work to the `double` version. Predict what happens, then run it and look at the trace. Why does the same line repeat? Fix it with a cast so that it prints 3.5, and use Question 24 to explain why the call chose itself.